2 Data transmission

In this chapter you will learn about:

- ★ types and methods of data transmission
  - how data is broken up into data packets before transmission
  - the structure of data packets (header, payload and trailer)
  - packet switching (including the role of the router in the process)
  - methods of data transmission (serial, parallel, simplex, half-duplex and full-duplex)
  - the Universal Serial Bus (USB)
- ★ methods of error detection
  - why error checking methods are needed
  - error checking methods following data transmission:
    - parity checks
    - checksum
    - echo check
  - use of check digits to detect data entry errors
  - use of automatic repeat requests (ARQs) to detect errors
- ★ encryption
  - the need for and the purpose of encryption
  - symmetric and asymmetric encryption
  - use of public and private keys.

Data is frequently transferred from one device to another. The two devices could be in the same building or thousands of kilometres away. Irrespective of the distance travelled, the transmission of data needs to be considered with respect to:

- how the data is transmitted
- how can errors following transmission be detected and can the data be recovered
- the role of encryption to make sure data that falls into the wrong hands can't be used

It is also important to consider ways of checking for errors in data once it has been entered into a computer.

2.1 Types and methods of data transmission

2.1.1 Data packets

Data packets are usually referred to simply as 'packets'

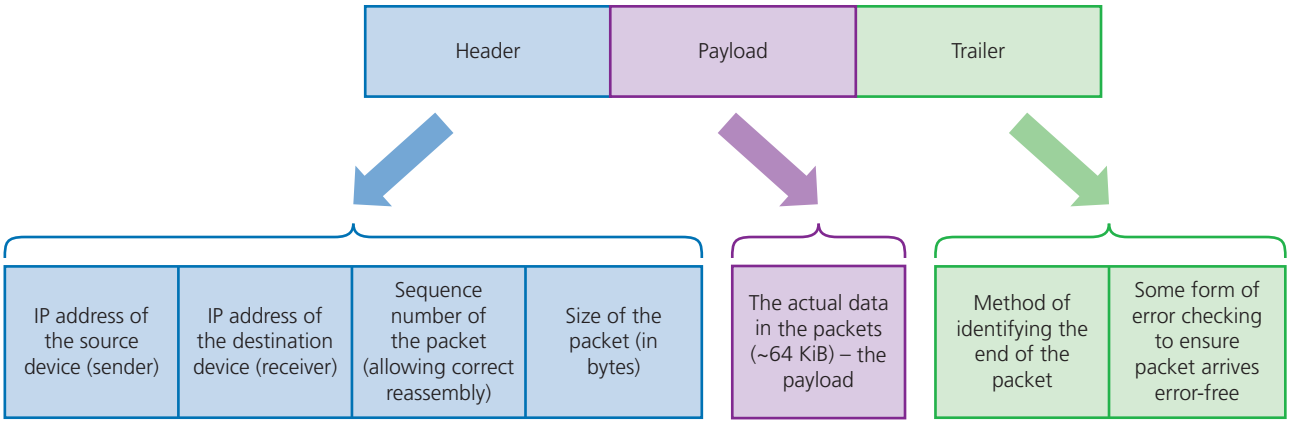
Data sent over long distances is usually broken up into **data packets** (sometimes called datagrams). The packets of data are usually quite small, typically 64 KiB, which are much easier to control than a long continuous stream of data. The idea of splitting up data in this way means each packet can be sent along a different route to its destination. This would clearly be of great benefit if a particular transmission route was out of action or very busy. The only obvious drawback of

splitting data into packets is the need to reassemble the data when it reaches its destination.

Packet structure

A typical packet is split up into:

- a packet header
- the payload
- a trailer.



▲ **Figure 2.1** Packet structure

For **each** packet, the **packet header** consists of:

- the IP address of the sending device
- the IP address of the receiving device
- the sequence number of the packet (this is to ensure that all the packets can be reassembled into the correct order once they reach the destination)
- packet size (this is to ensure the receiving station can check if all of the packets have arrived intact).

(Note: the header often also contains another value indicating how many packets there are in total for this transmission.)

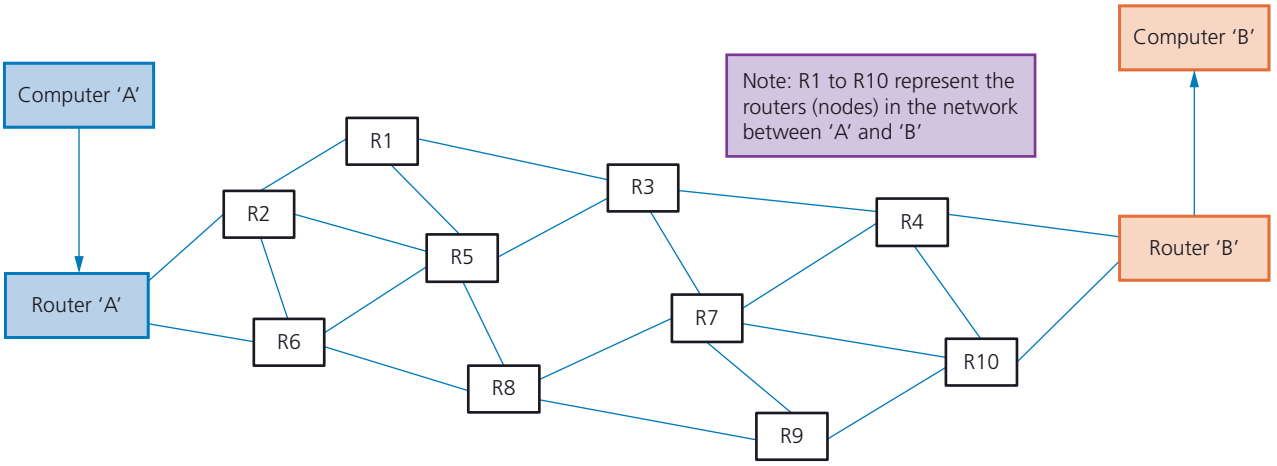
For **each** packet, the **payload** consists of the actual data being sent in the packet (this is usually about 64 KiB).

For **each** packet, the **packet trailer** consists of:

- some way of identifying the end of the packet; this is essential to allow each packet to be separated from each other as they travel from sending to receiving station
- an error checking method; **cyclic redundancy checks** (CRCs) are used to check data packets:
  - this involves the sending computer adding up all the 1-bits in the payload and storing this as a hex value in the trailer before it is sent
  - once the packet arrives, the receiving computer recalculates the number of 1-bits in the payload
  - the computer then checks this value against the one sent in the trailer
  - if the two values match, then no transmission errors have occurred; otherwise the packet needs to be re-sent.

Packet switching

Let us now consider what happens when a photograph, for example, is sent from computer 'A' to computer 'B'. The photograph will be split up into a number of packets before it is sent. There will be several possible routes for the packets, between computer 'A' (sender) and computer 'B' (receiver). Each stage in the route contains a **router**. A router receives a data packet and, based on the information in the header, decides where to send it next. For example:



▲ **Figure 2.2** Typical network showing possible routes between 'A' and 'B'

Packet switching is a method of data transmission in which a message is broken up into a number of packets. **Each** packet can then be sent independently from start point to end point. At the destination, the packets will need to be reassembled into their correct order (using the information sent in the header). At each stage in the transmission, there are **nodes** that contain a router. Each router will determine which route the packet needs to take, in order to reach its destination (the destination IP address is used in this part of the process).

Suppose our photograph (Figure 2.3) has been split up into five packets that have been sent in the following order:



▲ **Figure 2.3**

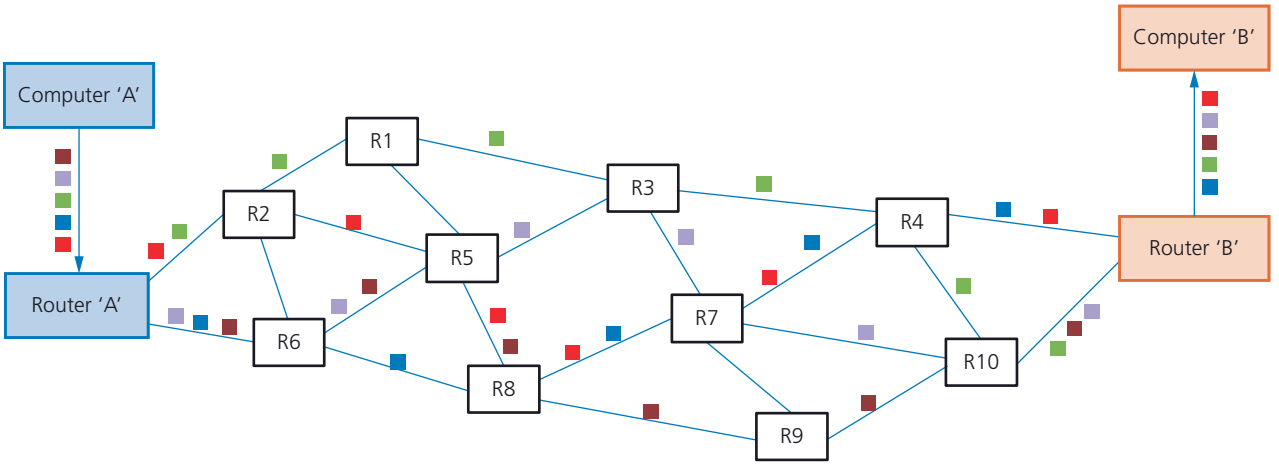
- each packet will follow its own path (route)
- routers will determine the route of each packet
- routing selection depends on the number of packets waiting to be processed at each node
- the shortest possible path **available** is always selected – this may not always be the shortest path that **could** be taken, since certain parts of the route may be too busy or not suitable
- unfortunately, packets can reach the destination in a different order to that in which they were sent.

Figure 2.5 shows one possible scenario. Notice the different paths taken by each packet from computer 'A' to computer 'B'. Also notice that the packets have arrived in a different order compared to the way they were sent, namely:



▲ **Figure 2.4**

Computer 'B' will now have to reassemble the packets into the original sequence.



▲ **Figure 2.5** Typical network showing possible paths taken by each packet

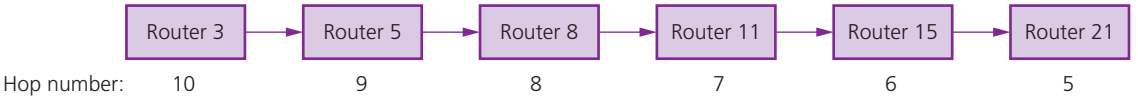
The benefits of packet switching are:

- there is no need to tie up a single communication line
- it is possible to overcome failed, busy or faulty lines by simply re-routing packets
- it is relatively easy to expand package usage
- a high data transmission rate is possible.

The drawbacks of packet switching include:

- packets can be lost and need to be re-sent
- the method is more prone to errors with **real-time streaming** (for example, a live sporting event being transmitted over the internet)
- there is a delay at the destination whilst the packets are being re-ordered.

Sometimes it is possible for packets to get lost because they keep 'bouncing' around from router to router and never actually reach their destination. Eventually the network would just grind to a halt as the number of **lost packets** mount up, clogging up the system. To overcome this, a method called **hopping** is used. A **hop number** is added to the header of each packet, and this number is reduced by 1 *every time it leaves a router* (Figure 2.6).



▲ **Figure 2.6** Hop numbers between routers

Each packet has a maximum hop number to start with. Once a hop number reaches zero, and the packet hasn't reached its destination, then the packet is deleted when it reaches the next router. The missing packets will then be flagged by the receiving computer and a request to re-send these packets will be made.

Find out more

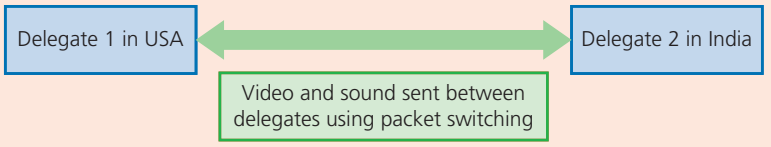
Another method of sending packets is called circuit switching. Find out how this differs to packet switching, and then re-draw Figure 2.5 showing the route the packets take when using circuit switching.

Advice

Hopping is not included on the syllabus but is included here for completeness (to help understand how packets can get lost).

Activity 2.1

1 Suppose a video conference is taking place between delegates in two different countries. Packet switching is being used to send video and sound data between the delegates:



▲ **Figure 2.7**

Describe:

i any potential problems with sound and video quality

ii how these problems could be caused.

2 Explain how packet switching could be used to download a large web page from a website.

3 a The trailer in a packet will use one form of error checking.

Explain what is meant by a cyclic redundancy check.

b The payload contains the following data:

```
11110000 10000011 00110011 00111111 11111110 11100011
```

Use this data to show how the receiving computer can verify that the received payload was error-free.

4 a Explain how it is possible for packets to be lost during their transmission across a network.

b Describe how it is possible for a system to deal with lost packets and prevent them from slowing down the transmission process.

c Explain why you think packet switching might improve data security.

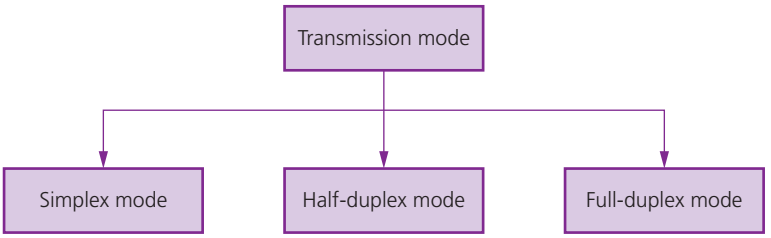
2.1.2 Data transmission

Data transmission can be either over a short distance (for example, computer to printer) or over longer distances (for example, from one computer to another in a global network). Essentially, three factors need to be considered when transmitting data:

- the direction of data transmission (for example, can data transmit in one direction only, or in both directions)
- the method of transmission (for example, how many bits can be sent at the same time)
- how will data be synchronised (that is, how to make sure the received data is in the correct order).

These factors are usually considered by a communication protocol.

Simplex, half-duplex and full-duplex



▲ **Figure 2.8** Transmission modes

Simplex data transmission

Simplex mode occurs when data can be sent in **ONE DIRECTION ONLY** (for example, from sender to receiver). An example of this would be sending data from a computer to a printer.

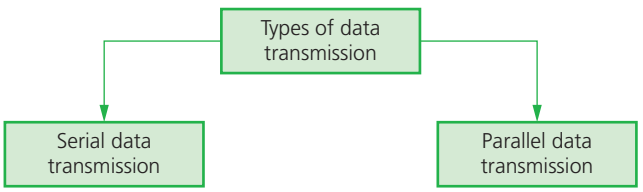
Half-duplex data transmission

Half-duplex mode occurs when data is sent in **BOTH DIRECTIONS** but **NOT AT THE SAME TIME** (for example, data can be sent from 'A' to 'B' and from 'B' to 'A' along the same transmission line, but they can't both be done at the same time). An example of this would be a walkie-talkie where a message can be sent in one direction only at a time; but messages can be both received and sent.

Full-duplex data transmission

Full-duplex mode occurs when data can be sent in **BOTH DIRECTIONS AT THE SAME TIME** (for example, data can be sent from 'A' to 'B' and from 'B' to 'A' along the same transmission line simultaneously). An example of this would be a broadband internet connection.

Serial and parallel data transmission



▲ **Figure 2.9** Types of data transmission

Serial data transmission occurs when data is sent **ONE BIT AT A TIME** over a **SINGLE WIRE/CHANNEL**. Bits are sent one after the other as a single stream.

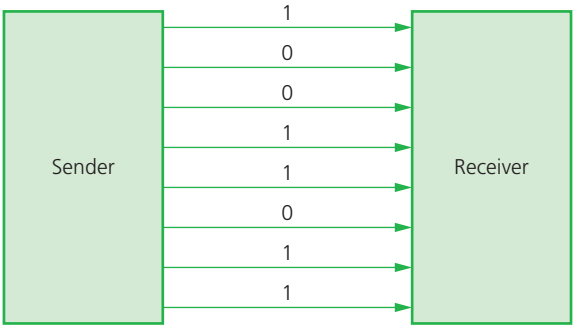


▲ **Figure 2.10** Serial data transmission

(Note: Serial data transmission can be simplex, half-duplex or full-duplex.)

Serial data transmission works well over long distances. However, the data is transmitted at a slower rate than parallel data transmission. Because only one channel/wire is used, data will arrive at its destination fully synchronised (i.e. in the correct order). An example of its use is when connecting a computer to a printer via a USB connection (see Section 2.1.3).

Parallel data transmission occurs when **SEVERAL BITS OF DATA** (usually one byte) are sent down **SEVERAL CHANNELS/WIRES** all at the same time. Each channel/wire transmits one bit:



▲ **Figure 2.11** Parallel data transmission

(Note: Parallel data transmission can be simplex, half-duplex or full-duplex.)

Parallel data transmission works well over short distances. Over longer distances (for example, over 20 metres), data can become **skewed** (that is, the data can arrive unsynchronised) and bits can arrive out of order. The longer the wire, the worse this can become. It is, however, a faster method of data transmission than serial. The internal circuits in a computer use parallel data transmission since the distance travelled between components is very short and high-speed transmission is essential.

Link

For more on data transmission within the CPU refer to Chapter 3.

Activity 2.2

1 Explain what is meant by:
   - i serial, half-duplex data transmission
   - ii parallel, full-duplex data transmission
   - iii serial, simplex data transmission.

2 Which types of data transmission are being described:
   - i data is sent one bit at a time in one direction only
   - ii data is being sent 8 bits at a time in one direction only
   - iii data is being sent 16 bits at a time in both directions simultaneously
   - iv data is sent one bit at a time in both directions simultaneously
   - v data is sent 16 bits at a time in one direction only?

Table 2.1 shows the comparison between serial and parallel data transmission.

▼ **Table 2.1** Comparison of serial and parallel data transmission methods

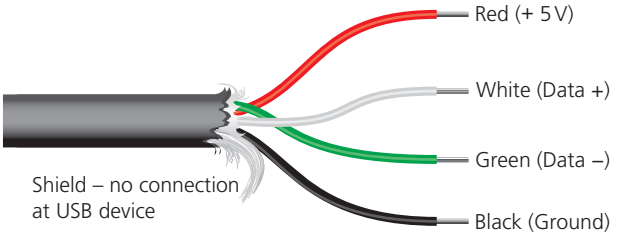
Serial	Parallel
less risk of external interference than with parallel (due to fewer wires)	faster rate of data transmission than serial
more reliable transmission over longer distances	works well over shorter distances (for example, used in internal pathways on computer circuit boards)
transmitted bits won't have the risk of being skewed (that is, out of synchronisation)	since several channels/wires used to transmit data, the bits can arrive out of synchronisation (skewed)
used if the amount of data being sent is relatively small since transmission rate is slower than parallel (for example, USB uses this method of data transmission)	preferred method when speed is important
used to send data over long distances (for example, telephone lines)	if data is time-sensitive, parallel is the most appropriate transmission method
less expensive than parallel due to fewer hardware requirements	parallel ports require more hardware, making them more expensive to implement than serial ports
	easier to program input/output operations when parallel used

2.1.3 Universal serial bus (USB)

Link

Also refer to Section 4.1 on software drivers regarding devices plugged into USB ports.

As the name suggests, the **universal serial bus (USB)** is a form of serial data transmission. USB is now the most common type of input/output port found on computers and has led to a standardisation method for the transfer of data between devices and a computer. It is important to note that USB allows both half-duplex and full-duplex data transmission.



▲ **Figure 2.12** Typical USB cable

As Figure 2.12 shows, the USB cable consists of a four-wired shielded cable, with two wires for power (red and black). The other two wires (white and green) are for data transmission. When a device is plugged into a computer using one of the USB ports:

- the computer automatically detects that a device is present (this is due to a small change in the voltage on the data signal wires in the USB cable)
- the device is automatically recognised, and the appropriate device driver software is loaded up so that the computer and device can communicate effectively
- if a new device is detected, the computer will look for the device driver that matches the device; if this is not available, the user is prompted to download the appropriate driver software (some systems do this automatically and the user will see a notice asking for permission to connect to the device website).